

PROYECTO FINAL DE CICLO

Implantación y monitorización de un sistema web para la gestión de un taller mecánico

CFGS Administración de Sistemas Informáticos en Red

Autor: Antonio Corredera Cubells

Curso académico: 2025/2026

Centro docente: [indicar centro]

Tutor/a individual: [indicar tutor/a]

La presente memoria documenta el análisis, diseño, implantación y validación de una solución web orientada a la gestión de un taller mecánico. El trabajo se ha enfocado desde la perspectiva del ciclo de Administración de Sistemas Informáticos en Red, por lo que no se limita a describir la aplicación, sino que explica también la infraestructura de ejecución, la seguridad, la observabilidad, el despliegue con contenedores y la propuesta de explotación en un entorno cloud.

ÍNDICE

ÍNDICE	2
ÍNDICE DE FIGURAS Y TABLAS	5
Índice de figuras	5
Índice de tablas.....	5
1. INTRODUCCIÓN	6
1.1. Módulos del CFGS ASIR a los que implica.....	6
1.2. Breve descripción del PFC	7
Objetivos del PFC	8
2. ESTUDIO PREVIO	10
2.1. Situación de partida	10
2.2. Necesidades detectadas.....	10
2.3. Soluciones existentes y solución propuesta	11
3. PLAN DE TRABAJO	13
4. DISEÑO.....	15
4.1. Arquitectura general.....	15
4.2. Modelo de datos	15
4.3. Flujo de reserva de cita.....	17
4.4. Diseño de redes y despliegue cloud	17
5. IMPLANTACIÓN	19
5.1. Aplicación web.....	19
5.2. Seguridad de aplicación.....	19
5.3. Docker Compose local.....	20
5.4. Despliegue en AWS.....	21
Reglas de entrada necesarias para abrir las UIs desde navegador	22
5.5. Monitorización y alertas	23
5.6. Simulador de tráfico	24

5.7. Requisitos no funcionales y criterios de calidad	25
5.8. Detalle de módulos funcionales	27
5.8.1. Módulo de autenticación y usuarios	27
5.8.2. Módulo de citas	28
5.8.3. Módulo de contenidos: noticias y consejos	28
5.8.4. Módulo de administración	29
5.9. Procedimiento de despliegue detallado	29
5.9.1. Seguimiento del primer despliegue en EC2	30
5.10. Copias de seguridad y restauración	32
5.11. Despliegues alternativos: Coolify y Dokploy	32
5.12. Operación diaria y mantenimiento	34
6. RECURSOS Y PRESUPUESTO	36
6.1. Recursos utilizados	36
6.2. Presupuesto anual orientativo	36
7. VALIDACIÓN Y PRUEBAS	38
7.1. Pruebas funcionales	38
7.2. Pruebas de despliegue y monitorización	38
7.3. Matriz de pruebas	39
7.4. Plan de demostración para la defensa	40
8. CONCLUSIONES	42
8.1. Grado de consecución de objetivos	42
8.2. Problemas encontrados	42
8.3. Mejoras futuras	43
9. ANEXOS Y DOCUMENTOS COMPLEMENTARIOS	45
9.1. Índice de anexos	45
Anexo A. Inventario de archivos principales	46
Inventario actualizado por carpetas	46

Anexo B. Comandos de despliegue.....	47
Comandos útiles en EC2 tras el bootstrap.....	47
Anexo C. Variables de entorno relevantes.....	48
Anexo D. Fragmentos de configuración y observabilidad.....	49
10. REFERENCIAS Y BIBLIOGRAFÍA.....	50

ÍNDICE DE FIGURAS Y TABLAS

Índice de figuras

Figura 1. Arquitectura general del sistema web y servicios de observabilidad.

Figura 2. Modelo de datos principal de la aplicación.

Figura 3. Flujo de reserva de cita desde la interfaz web.

Figura 4. Despliegue recomendado en AWS con EC2 y Docker Compose.

Figura 5. Flujo de monitorización y simulación de tráfico.

Figura 6. Flujo operativo del despliegue EC2 y consulta de credenciales.

Figura 7. Funcionamiento del simulador Apache JMeter dentro del proyecto.

Índice de tablas

Tabla 1. Relación entre módulos del CFGS ASIR y contenidos aplicados.

Tabla 2. Requisitos funcionales del sistema.

Tabla 3. Comparativa de soluciones existentes.

Tabla 4. Plan de trabajo y temporalización.

Tabla 5. Entidades principales de la base de datos.

Tabla 6. Servicios del despliegue Docker local.

Tabla 7. Servicios y puertos en despliegue AWS.

Tabla 8. Métricas y alertas implantadas.

Tabla 9. Recursos utilizados.

Tabla 10. Presupuesto anual orientativo.

Tabla 11. Grado de consecución de objetivos.

Tabla 12. Problemas encontrados y resolución.

Tabla 13. Propuestas de mejora.

1. INTRODUCCIÓN

1.1. Módulos del CFGS ASIR a los que implica

El proyecto integra competencias de varios módulos del ciclo formativo de Administración de Sistemas Informáticos en Red. La solución desarrollada parte de una aplicación web de gestión, pero su valor académico principal está en la implantación completa de un servicio: definición de base de datos, publicación mediante Apache y PHP, despliegue reproducible con Docker, operación en cloud, seguridad básica, monitorización técnica y pruebas de carga controladas.

Tabla 1. Relación entre módulos del CFGS ASIR y contenidos aplicados

Módulo	Aplicación en el PFC
Implantación de Aplicaciones Web	Despliegue de una aplicación PHP sobre Apache, gestión de sesiones, formularios, páginas públicas y privadas, estructura de rutas y publicación mediante Docker.
Gestión de Bases de Datos	Diseño de tablas MySQL, claves primarias y foráneas, relaciones 1:1 y 1:N, índices, restricciones de unicidad y persistencia de usuarios, citas, noticias y consejos.
Administración de Sistemas Gestores de Bases de Datos	Configuración de MySQL 8.0, inicialización mediante script SQL, usuarios de aplicación, acceso por variables de entorno, backups y posibilidad de migración futura a RDS.
Servicios de Red e Internet	Exposición de servicios HTTP, uso de puertos, redes Docker internas, resolución por nombres de servicio, rutas de monitorización y publicación mediante dominio.
Administración de Sistemas Operativos	Gestión de contenedores, volúmenes, logs, healthchecks, scripts de despliegue, permisos y resolución de incidencias en entornos Windows/Linux.
Seguridad y Alta Disponibilidad	Autenticación por roles, contraseñas cifradas, sesiones, CSRF en formularios críticos, prepared statements, sanitización, control de puertos y estrategia de alertas.

Módulo	Aplicación en el PFC
Planificación y Administración de Redes	Separación de redes de aplicación y monitorización, definición de Security Groups en AWS, acceso restringido a Prometheus, Grafana, Alertmanager y MySQL.
Lenguajes de Marcas y Sistemas de Gestión de Información	HTML5, CSS3, JavaScript, documentación Markdown, configuración YAML de Docker Compose y JSON de dashboards Grafana.
Empresa e Iniciativa Emprendedora	Análisis de una necesidad de negocio realista, digitalización de procesos de taller, presupuesto de implantación y evaluación de costes de explotación.
Inglés Técnico	Uso de documentación oficial de Docker, AWS, Prometheus, Grafana, PHP y MySQL, así como nomenclatura técnica habitual en herramientas cloud y de observabilidad.

Fuente: elaboración propia.

1.2. Breve descripción del PFC

El PFC consiste en la implantación de un sistema web para un taller mecánico que permite organizar la relación entre clientes, citas, contenidos informativos y administración interna. El sistema cubre tres perfiles principales: visitantes, usuarios registrados y administradores. Los visitantes pueden consultar información pública y solicitar una cita como invitados; los usuarios registrados pueden gestionar sus datos y consultar su historial; los administradores disponen de un panel para gestionar usuarios, agenda, noticias y consejos.

La solución está desarrollada con PHP, MySQL, HTML5, CSS3 y JavaScript. El acceso a datos se realiza mediante PDO y consultas preparadas. La autenticación utiliza sesiones PHP, regeneración de identificador de sesión y contraseñas almacenadas con bcrypt. Las operaciones de administración quedan restringidas por rol, lo que separa las acciones de usuario normal de las funciones de gestión del negocio.

Desde el punto de vista de ASIR, la parte más relevante es la implantación completa del servicio. El repositorio incluye Dockerfile, varios ficheros Docker Compose, guías de despliegue local, despliegue AWS, despliegue

Coolify/Dokploy, monitorización Prometheus/Grafana, Alertmanager, exporters, blackbox y un simulador de tráfico separado de la aplicación principal. Esta combinación permite demostrar no solo que la aplicación funciona, sino que puede operarse y vigilarse como un servicio real.

El sistema se plantea para una pequeña empresa de reparación de vehículos que necesita abandonar procedimientos manuales de reserva y comunicación. La aplicación centraliza citas y publicaciones, reduce la dependencia de llamadas telefónicas, facilita el acceso desde cualquier navegador y aporta indicadores técnicos para anticipar problemas de disponibilidad o rendimiento.

Objetivos del PFC

- Diseñar e implantar una aplicación web funcional para la gestión básica de un taller mecánico.
- Definir una base de datos relacional que mantenga usuarios, credenciales, citas, noticias y consejos con integridad referencial.
- Publicar la solución mediante contenedores Docker para que el despliegue sea reproducible en local y en cloud.
- Añadir una capa de monitorización que permita observar salud del sistema, uso de la aplicación, base de datos y métricas de negocio.
- Documentar una propuesta de despliegue profesional en AWS con costes, seguridad perimetral y estrategia de mantenimiento.

Tabla 2. Requisitos funcionales del sistema

Perfil	Funciones principales
Visitante	Consultar página de inicio y noticias, registrarse, iniciar sesión y solicitar una cita como invitado con datos de contacto.
Usuario registrado	Crear reservas, consultar citas propias, editar perfil, cambiar contraseña y mantener sus datos personales.
Administrador	Gestionar usuarios, citas, noticias, consejos, comprobar estadísticas básicas y acceder a herramientas de monitorización.

Perfil	Funciones principales
Operador técnico	Desplegar el sistema, revisar logs, comprobar métricas, validar healthchecks, ejecutar simulador de tráfico y restaurar backups.

Fuente: elaboración propia.

2. ESTUDIO PREVIO

2.1. Situación de partida

El escenario propuesto corresponde a un taller mecánico de tamaño pequeño o mediano que atiende tanto reparaciones planificadas como incidencias puntuales. En un funcionamiento tradicional, las reservas se realizan por teléfono, mensajes o presencialmente. Este modelo es suficiente en fases iniciales, pero genera problemas cuando aumenta el volumen de clientes: duplicidad de horarios, pérdida de información, poca trazabilidad y dependencia de una persona concreta para conocer la agenda.

La empresa también necesita publicar información de interés para sus clientes. Las noticias, campañas, avisos de mantenimiento y consejos técnicos suelen difundirse por canales externos o de forma informal, lo que dispersa la información. Una aplicación propia permite centralizar esos contenidos y mantener una relación más ordenada con el cliente.

Desde el punto de vista de sistemas, el taller no solo necesita una página web, sino un servicio estable. Si se produce una caída, un aumento de latencia o una saturación de base de datos, el negocio puede perder reservas. Por este motivo el proyecto incorpora observabilidad desde el inicio, con métricas, dashboards, alertas y pruebas de carga sintética controladas.

2.2. Necesidades detectadas

La primera necesidad es digitalizar el proceso de cita. El sistema debe permitir seleccionar fecha y hora, guardar el motivo de la visita y evitar que dos clientes ocupen el mismo tramo horario. Además, debe admitir reservas de usuarios autenticados y reservas de invitados, ya que no todos los clientes quieren registrarse antes de pedir una cita.

La segunda necesidad es separar responsabilidades. El cliente solo debe acceder a su información y a sus citas, mientras que el administrador necesita una vista completa para gestionar agenda, usuarios y publicaciones. Esta separación se resuelve con roles, sesiones y páginas protegidas.

La tercera necesidad es facilitar el despliegue. Un proyecto académico de ASIR debe poder instalarse de forma repetible, por lo que Docker Compose es una solución adecuada: define servicios, redes, volúmenes, puertos, variables de entorno y dependencias entre contenedores.

La cuarta necesidad es operar la aplicación con visibilidad. Prometheus recoge métricas, Grafana las muestra en paneles, Alertmanager permite notificaciones y blackbox comprueba rutas HTTP desde fuera del proceso de la aplicación. Con ello se detectan problemas antes de que afecten al cliente final.

2.3. Soluciones existentes y solución propuesta

Tabla 3. Comparativa de soluciones existentes

Alternativa	Ventajas	Limitaciones frente al PFC
Agenda manual o hoja de cálculo	Coste inicial bajo y aprendizaje rápido.	No hay acceso web, control de roles, trazabilidad, validaciones ni monitorización.
Google Calendar compartido	Permite organizar citas y avisos básicos.	No integra noticias, perfiles, panel de administración ni métricas de negocio propias.
SaaS de gestión de talleres	Producto maduro con soporte comercial.	Coste recurrente, dependencia de proveedor y menor valor académico para ASIR.
WordPress con plugins de reservas	Rápido de publicar y con ecosistema amplio.	Mayor superficie de ataque por plugins, menor control del backend y de la arquitectura.
Solución propuesta PHP/MySQL Docker	Control completo, despliegue reproducible, monitorización integrada y adaptación al caso de estudio.	Requiere administración técnica y mantenimiento por parte del equipo responsable.

Fuente: elaboración propia.

Se elige una solución propia porque permite demostrar competencias del ciclo de forma integrada. La aplicación no depende de un proveedor externo, utiliza tecnologías conocidas en entornos LAMP y se acompaña de infraestructura de

contenedores, documentación, métricas y una propuesta de explotación cloud. Esta decisión es coherente con el objetivo formativo: mostrar que el alumno puede diseñar, desplegar y mantener un servicio completo.

3. PLAN DE TRABAJO

La planificación se ha distribuido en fases progresivas para reflejar el ciclo completo de un proyecto de sistemas: análisis, diseño, construcción, despliegue, monitorización, pruebas y documentación. Las horas son aproximadas y se ajustan a la naturaleza del módulo de Proyecto, aunque el desarrollo real del repositorio contiene más iteraciones técnicas que las recogidas de forma resumida en esta tabla.

Tabla 4. Plan de trabajo y temporalización

Semana	Tareas realizadas	Horas
Lunes 03/03/26 - Viernes 07/03/26	Análisis de requisitos, definición de escenario de taller, revisión de módulos ASIR implicados y estructura inicial del repositorio.	5
Lunes 10/03/26 - Viernes 14/03/26	Diseño del modelo de datos, tablas principales, relaciones, restricciones y usuarios iniciales.	5
Lunes 17/03/26 - Viernes 21/03/26	Desarrollo de autenticación, registro, login, sesiones, roles y estructura común de cabecera/pie.	6
Lunes 24/03/26 - Viernes 28/03/26	Implementación de gestión de citas, calendario, API JSON, validación de horarios y reservas de invitados.	6
Lunes 31/03/26 - Viernes 04/04/26	Implementación de noticias, consejos, subida de imágenes y panel de administración.	5
Lunes 07/04/26 - Viernes 11/04/26	Contenerización con Docker, variables de entorno, volúmenes, redes, healthchecks e inicialización de MySQL.	6
Lunes 14/04/26 - Viernes 18/04/26	Monitorización con Prometheus, Grafana, exporters, endpoint de métricas PHP, alertas y dashboards.	6

Semana	Tareas realizadas	Horas
Lunes 21/04/26 - Viernes 25/04/26	Simulador de tráfico, pruebas de carga controladas, documentación de despliegue y revisión de seguridad.	5
Lunes 28/04/26 - Viernes 02/05/26	Redacción de memoria, anexos, presupuesto, conclusiones y preparación de defensa.	6
Total	Planificación, implantación y documentación del PFC.	50

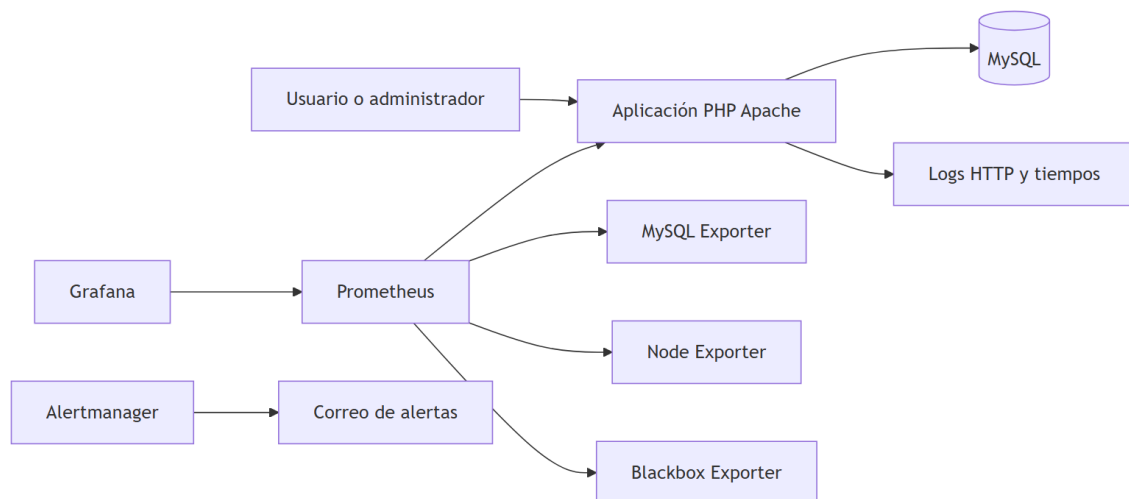
Fuente: elaboración propia.

4. DISEÑO

4.1. Arquitectura general

La arquitectura se organiza alrededor de una aplicación PHP ejecutada en Apache, una base de datos MySQL y un conjunto de servicios auxiliares de monitorización. En desarrollo, Docker Compose levanta todos los componentes en una misma máquina. En AWS, el compose de producción publica la aplicación web y también puede publicar las interfaces de Grafana, Prometheus, Alertmanager y la UI del simulador para acceder desde el navegador. Esa publicación depende de dos capas: el bind de Docker, configurado con `MONITORING_UI_HOST_BIND=0.0.0.0`, y las reglas de entrada del Security Group, que deben limitarse a la IP o red administrativa siempre que sea posible.

Figura 1. Arquitectura general del sistema web y servicios de observabilidad.

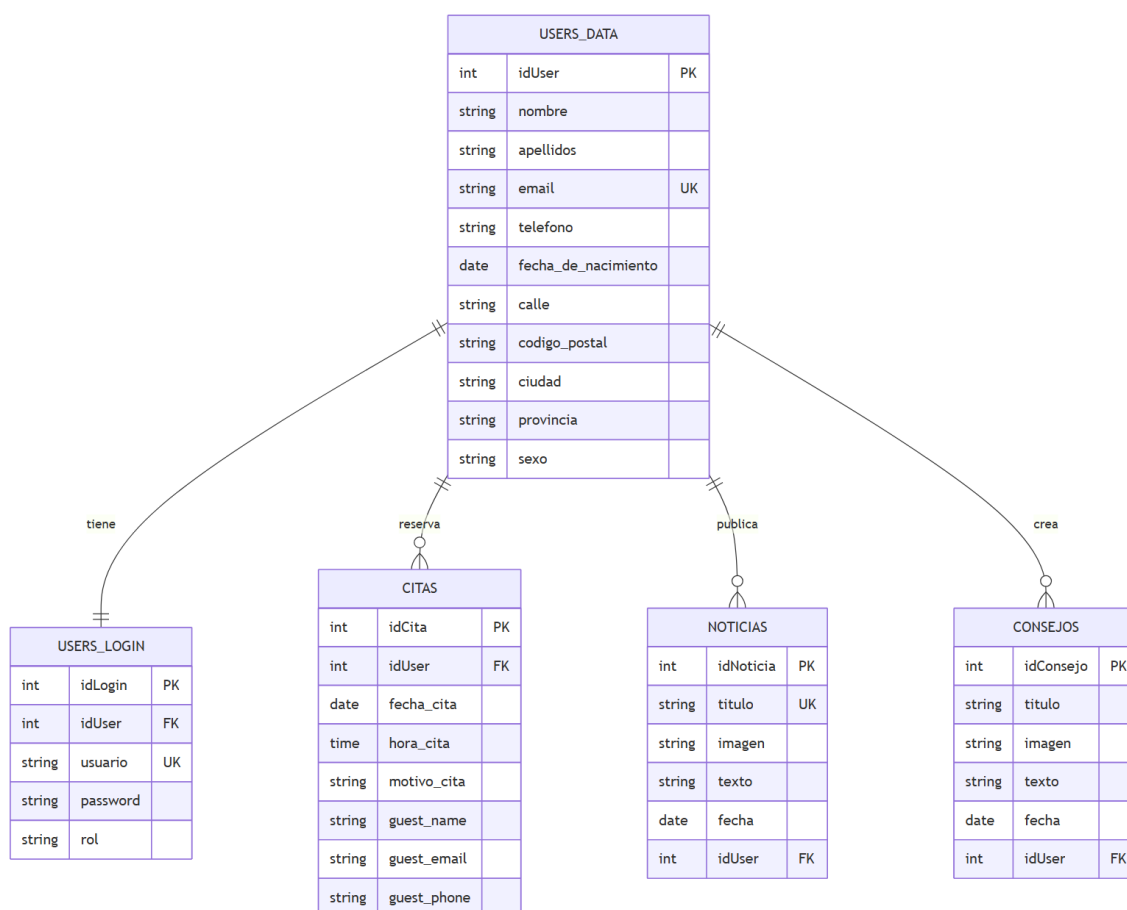


Fuente: elaboración propia a partir del repositorio del proyecto.

4.2. Modelo de datos

La base de datos se denomina `trabajo_final_php` y utiliza MySQL con tablas InnoDB. El modelo separa datos personales y credenciales para evitar mezclar información de perfil con autenticación. Las citas pueden estar asociadas a un usuario registrado o a un invitado. Las noticias y consejos están vinculados a usuarios administradores mediante claves foráneas.

Figura 2. Modelo de datos principal de la aplicación.



Fuente: elaboración propia a partir del repositorio del proyecto.

Tabla 5. Entidades principales de la base de datos

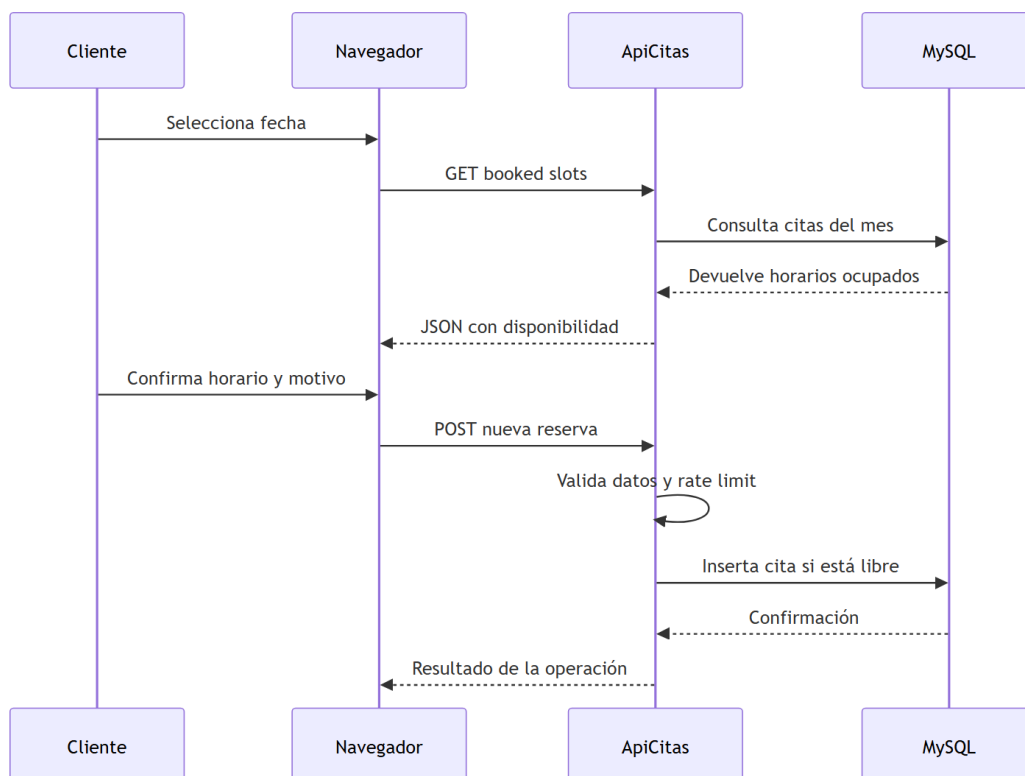
Tabla	Propósito	Relaciones
users_data	Datos personales y de contacto.	Relación 1:1 con users_login y 1:N con citas, noticias y consejos.
users_login	Credenciales, usuario y rol.	Clave foránea idUser con eliminación en cascada.
citas	Reservas con fecha, hora, motivo y datos de invitado.	Puede vincularse a users_data o registrar datos de invitado.
noticias	Publicaciones del taller con imagen y texto.	Cada noticia se asocia al usuario administrador creador.
consejos	Consejos técnicos o contenido informativo.	Cada consejo se asocia al usuario que lo creó.

Fuente: elaboración propia.

4.3. Flujo de reserva de cita

El flujo de reserva se diseñó para ser sencillo. El usuario selecciona una fecha, la interfaz consulta la API para recuperar horarios ocupados y solo permite confirmar los tramos disponibles. Si el cliente está autenticado, la cita se vincula a su identificador; si no lo está, se exigen datos básicos de contacto. La API valida formato de fecha y hora, comprueba que la cita no esté en el pasado, aplica una limitación básica de frecuencia y evita duplicados por fecha y hora.

Figura 3. Flujo de reserva de cita desde la interfaz web.



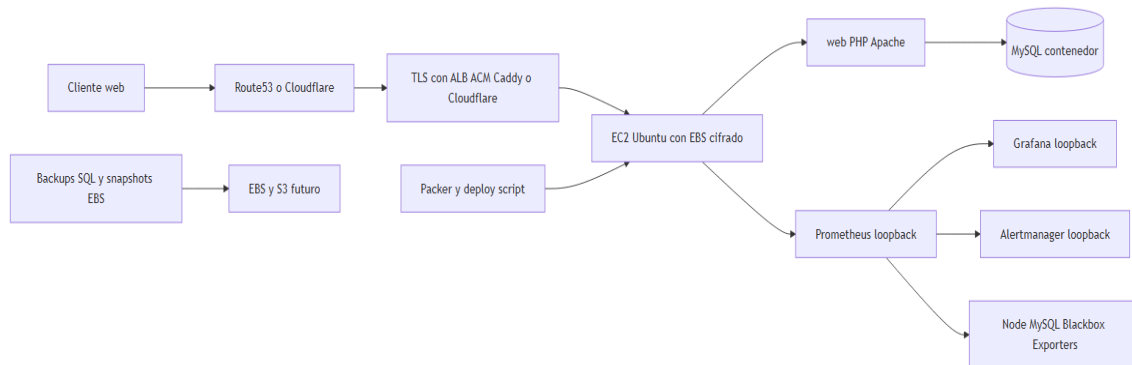
Fuente: elaboración propia a partir del repositorio del proyecto.

4.4. Diseño de redes y despliegue cloud

El diseño Docker local diferencia la red de aplicación y la red de monitorización. El servicio web se conecta a ambas porque necesita hablar con MySQL y, al mismo tiempo, exponer métricas a Prometheus. MySQL no debe exponerse públicamente en producción. En AWS, el compose recomendado publica la aplicación web en el puerto 80 y publica las interfaces de monitorización en puertos concretos para que puedan abrirse desde el navegador cuando el Security Group lo permita. Los exporters técnicos, como cAdvisor y Telegraf,

quedan ligados a 127.0.0.1 o a la red interna, porque solo los debe consultar Prometheus.

Figura 4. Despliegue recomendado en AWS con EC2 y Docker Compose.



Fuente: elaboración propia a partir del repositorio del proyecto.

5. IMPLANTACIÓN

5.1. Aplicación web

La aplicación se estructura en páginas PHP para navegación pública, autenticación, perfil, citas, noticias y administración. Los archivos comunes de cabecera, pie y funciones auxiliares se ubican en `includes/`, mientras que la conexión a base de datos se centraliza en `config/database.php`. Esta separación reduce duplicación y facilita cambios en configuración.

El registro crea una entrada en `users_data` y otra en `users_login` dentro de una transacción. Antes de insertar, valida campos obligatorios, formato de correo, teléfono español y código postal. La contraseña se guarda mediante `password_hash`, evitando texto plano. El login usa `password_verify`, regenera el identificador de sesión y genera un nuevo token CSRF.

La gestión de citas se apoya en una interfaz con calendario y llamadas fetch a `api/citas_api.php`. La API responde con JSON y registra métricas de cada petición mediante `includes/metrics_logger.php`, de manera que la actividad de reservas también queda reflejada en Prometheus.

La administración incluye CRUD de usuarios, citas, noticias y consejos. Las noticias permiten subir imágenes con validación de tipo MIME, extensión, tamaño máximo y bytes mágicos. Los consejos técnicos pueden cargarse desde datos semilla y editarse desde un panel de administración.

5.2. Seguridad de aplicación

La autenticación se basa en sesiones PHP. Las páginas protegidas comprueban la existencia de `user_id` y las zonas de administración verifican el rol admin. Esta lógica evita que un usuario normal acceda a funciones de mantenimiento.

La protección frente a inyección SQL se implementa con PDO y sentencias preparadas. El uso de `PDO::ATTR_EMULATE_PREPARES => false` favorece prepared statements nativos del motor. Los datos de salida se tratan con `htmlspecialchars` y los datos de entrada se limpian con `trim` y `strip_tags` en funciones auxiliares.

El proyecto incorpora tokens CSRF en formularios sensibles como login y registro. En la API de citas se añade una limitación básica de frecuencia para reducir spam de reservas. En producción se recomienda complementar estas medidas con HTTPS obligatorio, cookies seguras, cabeceras de seguridad y políticas de retención de datos personales.

5.3. Docker Compose local

El despliegue local se define en docker-compose.yml. Este fichero crea la aplicación web, MySQL, Prometheus, Alertmanager, Grafana, Node Exporter, MySQL Exporter, cAdvisor, Telegraf, blackbox-exporter y, opcionalmente, el perfil traffic con simulador y UI. Los volúmenes persistentes mantienen datos de MySQL, Prometheus, Grafana y Alertmanager, además de imágenes subidas y logs de la aplicación.

Tabla 6. Servicios del despliegue Docker local

Servicio	Imagen o build	Función
web	Build desde Dockerfile	Apache + PHP 8.2, aplicación web y endpoint /metrics.php.
mysql	mysql:8.0	Base de datos relacional con volumen persistente e inicialización SQL.
prometheus	prom/prometheus:v3.11.3	Scraping de métricas y evaluación de reglas de alerta.
grafana	grafana/grafana:13.0.1	Dashboards técnicos y de negocio provisionados desde archivos.
alertmanager	prom/alertmanager:v0.32.1	Agrupación y envío de alertas por correo SMTP.
node-exporter	prom/node-exporter:v1.11.1	Métricas del sistema operativo anfitrión.
mysqld-exporter	prom/mysqld-exporter:v0.19.0	Métricas de rendimiento de MySQL.

Servicio	Imagen o build	Función
blackbox-exporter	prom/blackbox-exporter:v0.26.0	Probes HTTP sintéticos sobre rutas críticas.
traffic-simulator	Build desde docker/traffic-simulator	Generación de tráfico HTTP controlado contra la aplicación.
traffic-simulator-ui	Build desde docker/traffic-simulator-ui	Interfaz web separada para arrancar/parar simulaciones.

Fuente: elaboración propia.

5.4. Despliegue en AWS

Para producción se propone usar una instancia EC2 con Docker Compose y volúmenes persistentes. El fichero docker-compose.aws.yml evita montar todo el código como bind-mount, no publica MySQL y separa los servicios por perfiles: web y mysql como base, monitoring para Prometheus/Grafana/Alertmanager/exporters, y traffic para Apache JMeter y su UI. En la configuración AWS actual, las UIs de monitorización se publican para navegador mediante `MONITORING_UI_HOST_BIND=0.0.0.0`; por eso la protección real debe estar en el Security Group, usando `MONITORING_SG_CIDR` con la IP administrativa en vez de abrir `0.0.0.0/0` en producción.

La automatización se apoya en Packer para construir una AMI base con Docker Engine y plugin Compose, y en `scripts/ec2-user-data-bootstrap.sh` para Amazon Linux 2023. El bootstrap instala Docker, Compose y Buildx, crea swap, clona el repositorio en `/opt/taller_mecanico_asir`, genera secretos cuando detecta valores de ejemplo y ejecuta `scripts/deploy_aws_docker.sh`. El script de despliegue realiza preflight de rutas, memoria y disco, backup opcional de MySQL, build/pull/up del compose, comprobaciones de salud y smoke tests de HTTP, Grafana, Prometheus, Alertmanager y JMeter cuando el perfil traffic está activo.

Tabla 7. Servicios y puertos en despliegue AWS

Componente	Exposición recomendada	Justificación
Aplicación web	80/443 mediante proxy, ALB o Cloudflare	Servicio público para usuarios y clientes del taller.
MySQL	Solo red Docker interna	Evita exposición directa de base de datos.
Prometheus	IP_PUBLICA:9090, restringido por Security Group	Interfaz consultable desde navegador; no tiene autenticación fuerte propia, así que debe restringirse por Security Group o proxy seguro.
Grafana	IP_PUBLICA:3000, restringido por Security Group	Panel de operación accesible desde navegador si el Security Group permite el puerto; en producción se recomienda limitarlo a la IP del administrador.
Alertmanager	IP_PUBLICA:9093, restringido por Security Group	Interfaz de alertas consultable desde navegador si se abre el puerto; debe protegerse igual que Prometheus.
Exporters	Solo red interna	Deben ser consumidos por Prometheus, no por Internet.

Fuente: elaboración propia.

Reglas de entrada necesarias para abrir las UIs desde navegador

En la instancia EC2 hay dos cosas distintas: Docker debe publicar el puerto y AWS debe permitir la entrada por el Security Group. Si una de las dos falla, el

servicio puede estar sano dentro del servidor pero no abrirse desde el navegador.

Puertos recomendados para laboratorio o defensa: 80 para la aplicación web, 3000 para Grafana, 9090 para Prometheus, 9093 para Alertmanager y 8890 para la UI del simulador JMeter. Estos puertos deben abrirse solo desde la IP del administrador cuando sea posible, por ejemplo con `MONITORING_SG_CIDR=TU_IP/32`.

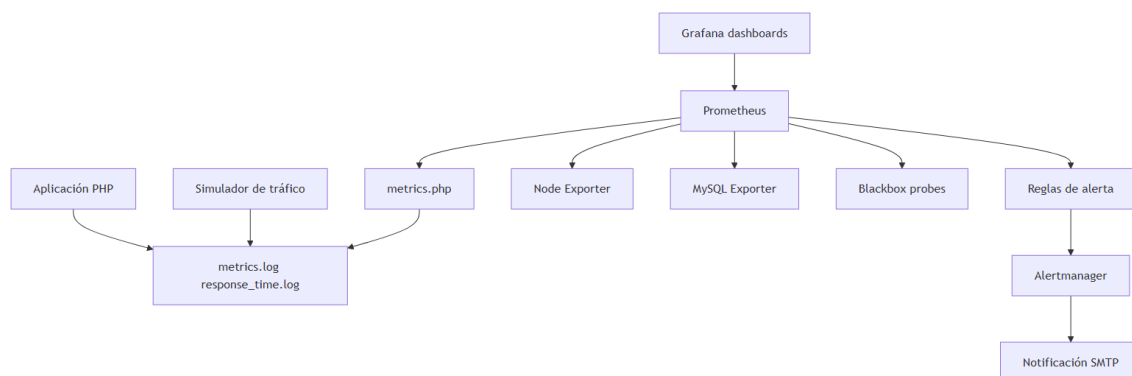
Puertos que no se deben abrir a Internet: MySQL 3306, cAdvisor 8080, Telegraf 9273, Node Exporter 9100, MySQL Exporter 9104 y Blackbox Exporter 9115. Son servicios técnicos pensados para Prometheus o para la red interna, no para usuarios finales.

Si Grafana o Prometheus no abren en el navegador, se revisa primero que el despliegue haya terminado, después `docker ps`, después `docker compose ps -a`, y por último las reglas del Security Group. En AWS el propio deploy intenta abrir 3000, 9090, 9093 y 8890 si la instancia tiene permisos IAM y `awscli` instalado.

5.5. Monitorización y alertas

La observabilidad combina métricas generadas por la aplicación, exporters del sistema y probes sintéticos. El endpoint `metrics.php` expone métricas de negocio, salud de base de datos, peticiones HTTP y tiempos de respuesta. Prometheus las recoge periódicamente, evalúa reglas de alerta y Grafana las presenta en paneles preconfigurados.

Figura 5. Flujo de monitorización y simulación de tráfico.



Fuente: elaboración propia a partir del repositorio del proyecto.

Tabla 8. Métricas y alertas implantadas

Área	Métricas o alertas	Utilidad
Aplicación	Requests por método, código HTTP, fuente app/simulator y percentiles de respuesta.	Detectar errores, latencia y efecto de pruebas de carga.
Negocio	Usuarios totales, citas totales, citas futuras/pasadas, noticias y consejos.	Medir actividad funcional del taller.
Base de datos	Conexiones, salud de conexión, consultas, slow queries y disponibilidad del exporter.	Detectar saturación o caída de MySQL.
Sistema	CPU, memoria, disco y red.	Vigilar recursos de la instancia o host Docker.
Blackbox	Probe success sobre rutas /, /login.php, /citaciones.php y /metrics.php.	Confirmar disponibilidad desde fuera del proceso PHP.
Alertmanager	Agrupación por severidad y envío SMTP.	Reducir ruido y notificar incidencias críticas.

Fuente: elaboración propia.

5.6. Simulador de tráfico

El simulador se ha separado de la aplicación principal para evitar que una herramienta de pruebas forme parte del panel de administración. Con el perfil traffic, Docker Compose inicia un worker con API HTTP interna y una UI independiente. La UI actúa como proxy server-side y no expone el token de control al navegador. Las métricas del simulador se escriben en el mismo formato que la aplicación, identificadas con source=simulator, lo que permite diferenciarlas en Grafana.

La herramienta incluye controles para evitar abuso: token de control, política de objetivos internos y externos, confirmación para URLs no internas, rechazo de rangos privados salvo configuración de laboratorio y verificación TLS por defecto.

Estas medidas son importantes porque una herramienta de carga puede ser peligrosa si se expone sin restricciones.

Apache JMeter no es una página más del taller. Es una herramienta de pruebas que crea muchas peticiones HTTP para comprobar cómo responde la aplicación cuando hay varios usuarios simulados. En este proyecto se ejecuta en modo CLI, sin interfaz gráfica propia, dentro del contenedor traffic-simulator. Cuando el operador inicia una prueba desde la UI, la UI no llama directamente a JMeter. Primero envía la petición al API interno del worker, protegido por `SIMULATOR_CONTROL_TOKEN`. El worker genera un plan `traffic-test.jmx`, un fichero `routes.csv` con las rutas y pesos, ejecuta `jmeter -n -t traffic-test.jmx -l results.jtl -j jmeter.log`, y guarda también `stdout.log` para diagnóstico.

JMeter escribe los resultados en `results.jtl`. El runner del proyecto lee ese fichero y lo convierte al formato que ya entiende `metrics.php`: `logs/metrics.log` recibe líneas como `GET 200 /ruta source=simulator` y `logs/response_time.log` recibe tiempos de respuesta. Así Grafana puede separar el tráfico real de la aplicación, `source=app`, del tráfico de prueba, `source=simulator`.

Uso básico: en local se ejecuta `docker compose --profile traffic up -d` y se abre `http://localhost:8890`. En AWS, el perfil `traffic` viene activado por `COMPOSE_PROFILES=monitoring,traffic` en `.env.aws.example` y la UI se abre en `http://IP_PUBLICA:8890` si el Security Group permite ese puerto.

Figura 7. Funcionamiento del simulador Apache JMeter dentro del proyecto.

```

```mermaid
flowchart TD
 operador["Operador"] --> uiSimulador["UI simulador puerto 8890"]
 uiSimulador --> apiProxy["api.php proxy server-side"]
 apiProxy --> worker["traffic-simulator API 8085 interno"]
 worker --> jmx["Genera traffic-test.jmx y routes.csv"]
 jmx --> jmeter["Apache JMeter CLI"]
 jmeter --> jtl["results.jtl y jmeter.log"]
 jtl --> logs["metrics.log con source=simulator"]
 logs --> prometheus["Prometheus lee metrics.php"]
 prometheus --> grafana["Grafana muestra carga simulada"]
```

```

Fuente: elaboración propia a partir del funcionamiento del contenedor `traffic-simulator`.

5.7. Requisitos no funcionales y criterios de calidad

Además de las funciones visibles para el usuario, el proyecto se ha diseñado teniendo en cuenta requisitos no funcionales. En administración de sistemas, estos requisitos son especialmente importantes porque determinan si una aplicación puede mantenerse en el tiempo, recuperarse ante fallos y adaptarse a cambios de infraestructura.

La disponibilidad se aborda con healthchecks, separación de servicios y monitorización. El servicio web no debe considerarse operativo hasta que MySQL esté aceptando conexiones. Del mismo modo, la disponibilidad externa no se mide solo por el estado del contenedor, sino mediante probes HTTP que simulan peticiones reales a rutas relevantes.

La mantenibilidad se apoya en documentación, variables de entorno y ficheros de despliegue separados. El uso de `.env.example`, `.env.aws.example` y varios `compose` evita mezclar configuración de desarrollo con producción. Esta decisión facilita que otro administrador pueda revisar el proyecto y adaptarlo a un entorno concreto sin modificar el código fuente.

La portabilidad se consigue con Docker. Aunque el desarrollo se haya realizado en Windows, el stack puede ejecutarse en Linux, macOS o en una instancia EC2. Las diferencias entre sistemas operativos se documentan, especialmente en componentes de monitorización que dependen del host, como Node Exporter, cAdvisor o Telegraf.

La seguridad se plantea por capas: aplicación, base de datos, contenedores, red y operación. Ninguna medida aislada es suficiente. El control de roles protege funciones internas, las consultas preparadas reducen riesgo de inyección SQL, los secretos deben estar fuera del repositorio y los servicios de administración no deben exponerse directamente a Internet.

Tabla 9. Requisitos no funcionales

| Categoría | Decisión de diseño | Evidencia en el proyecto |
|----------------|-----------------------------|---|
| Disponibilidad | Healthchecks y probes HTTP. | <code>docker-compose.yml</code> , <code>docker-compose.aws.yml</code> , <code>blackbox.yml</code> . |

| Categoría | Decisión de diseño | Evidencia en el proyecto |
|-----------------------|--|---|
| Mantenibilidad | Documentación técnica y ficheros de configuración por entorno. | docs/, .env.example, .env.aws.example. |
| Portabilidad | Contenedores Docker y Compose. | Dockerfile, docker-compose*.yaml. |
| Seguridad | Roles, sesiones, secretos por entorno y puertos restringidos. | includes/functions.php, config/database.php, compose AWS. |
| Observabilidad | Métricas, logs, dashboards y alertas. | monitoring/, includes/metrics_logger.php. |
| Escalabilidad inicial | Separación lógica de servicios y propuesta futura RDS/CI-CD. | docs/AWS_DOCKER_DEPLOYMENT.md y conclusiones. |

Fuente: elaboración propia.

5.8. Detalle de módulos funcionales

5.8.1. Módulo de autenticación y usuarios

El módulo de autenticación cubre registro, inicio de sesión, cierre de sesión y control de rol. El registro inserta datos personales y credenciales en tablas separadas. Esta separación permite que la información de acceso tenga su propio ciclo de validación y que el perfil del usuario pueda evolucionar sin modificar la tabla de login.

El inicio de sesión comprueba el nombre de usuario y valida la contraseña con `password_verify`. Al autenticarse, se guardan en sesión el identificador de usuario, el nombre de usuario y el rol. La regeneración del identificador de sesión reduce el riesgo de fijación de sesión, una medida básica pero importante en aplicaciones PHP.

El control de roles se centraliza en funciones auxiliares. Esta decisión evita repetir lógica en cada página y facilita auditorías: las páginas administrativas deben llamar a `isAdmin` o a funciones equivalentes antes de mostrar contenido

sensible. En una evolución futura se podría ampliar a un sistema de permisos más granular.

5.8.2. Módulo de citas

El módulo de citas es el núcleo funcional del sistema. Gestiona la disponibilidad horaria, las reservas de usuarios registrados y las reservas de invitados. La decisión de permitir invitados responde a un caso real: muchos clientes desean reservar rápido sin crear una cuenta, pero el taller necesita un nombre, correo y teléfono para poder contactar.

La interfaz muestra un calendario mensual y consulta por AJAX los tramos ocupados. El backend devuelve un mapa de fechas y horas reservadas. El navegador construye botones de horario y deshabilita los ocupados. Aunque esta validación mejora la experiencia, la validación definitiva siempre se repite en servidor antes de insertar la cita.

La API comprueba que la fecha no sea pasada, que el formato de hora sea válido y que el horario siga libre. Si otro usuario reserva el mismo tramo entre la consulta y el envío, la API responde con conflicto. Esta comprobación en servidor es esencial para evitar inconsistencias.

5.8.3. Módulo de contenidos: noticias y consejos

El módulo de noticias permite al taller publicar información corporativa, campañas, avisos y novedades. Las noticias incluyen título, texto, fecha, autor e imagen. En administración se valida que el título sea único y que el fichero subido sea una imagen permitida.

El módulo de consejos complementa las noticias con contenido técnico o educativo. Puede utilizarse para publicar recomendaciones de mantenimiento, revisiones periódicas, presión de neumáticos o preparación del vehículo para ITV. Desde el punto de vista de negocio, este módulo mejora la comunicación con clientes y aporta valor más allá de la reserva de citas.

Ambos módulos demuestran gestión de contenido en una aplicación propia. Aunque no son un CMS completo, cubren operaciones CRUD y subida o referencia de imágenes, con una complejidad adecuada para un PFC de ASIR centrado en implantación y administración del servicio.

5.8.4. Módulo de administración

El panel de administración reúne funciones que no deben estar disponibles para usuarios normales. Desde este entorno se consultan estadísticas básicas, se gestionan usuarios, se revisa la agenda, se editan noticias y se cargan consejos. Esta separación ayuda a explicar el control de acceso por roles y los riesgos de exposición indebida.

La vista administrativa también sirve como punto de operación funcional. Un responsable del taller puede comprobar citas pendientes, editar datos de usuarios o publicar avisos sin acceder directamente a la base de datos. Esto evita operaciones manuales peligrosas y reduce la necesidad de conocimientos técnicos en el día a día del negocio.

Desde la perspectiva de sistemas, el panel no sustituye a Grafana ni a Prometheus. La administración funcional se realiza en la aplicación; la administración técnica se realiza en herramientas de observabilidad. Mantener esta separación evita mezclar operaciones de negocio con operaciones de infraestructura.

5.9. Procedimiento de despliegue detallado

El despliegue local comienza preparando variables de entorno. El administrador copia `.env.example` a `.env`, revisa contraseñas, puertos y parámetros de monitorización. Después ejecuta `docker compose up -d --build`. Docker construye la imagen de la aplicación, descarga imágenes de terceros, crea redes, volúmenes y arranca servicios con healthchecks.

El primer arranque de MySQL ejecuta `database/database.sql` porque el volumen está vacío. Si el volumen ya existe, MySQL no vuelve a ejecutar scripts de inicialización. Por ello se documenta la diferencia entre despliegue fresco, migración y restauración. Esta es una incidencia habitual en entornos Docker y debe explicarse en la memoria de ASIR.

Tras levantar el stack, se verifica la aplicación web, el endpoint de métricas y los targets de Prometheus. En Grafana se comprueba que el datasource Prometheus está provisionado y que el dashboard principal carga datos. Si se

activa el perfil de tráfico, se valida que el simulador, la UI y la aplicación comparten el mismo volumen de logs.

En AWS, el procedimiento cambia. La instancia EC2 se prepara con Docker, ya sea manualmente, mediante una AMI generada con Packer o mediante el user data de Amazon Linux 2023. El repositorio se clona en `/opt/taller_mecanico_asir`, se crea o revisa `.env`, se rotan secretos y se ejecuta `scripts/deploy_aws_docker.sh`. Las UIs de monitorización se publican en puertos de navegador, pero el acceso debe quedar restringido por Security Group o por un proxy seguro.

Tabla 10. Checklist de despliegue

| Paso | Comprobación | Resultado esperado |
|----------------------------|--|---|
| Preparar <code>.env</code> | Contraseñas, puertos, SMTP y token del simulador. | Sin secretos por defecto en producción. |
| Construir imágenes | <code>docker compose build</code> o <code>up -d --build</code> . | Imagen web construida sin errores. |
| Arrancar MySQL | Healthcheck <code>mysqladmin ping</code> . | Estado healthy antes de web. |
| Arrancar web | Petición HTTP a <code>/</code> . | Respuesta 200 o página inicial visible. |
| Validar métricas | Acceder a <code>/metrics.php</code> . | Formato Prometheus sin HTML ni errores. |
| Validar Prometheus | Targets en estado UP. | Scraping correcto de app y exporters. |
| Validar Grafana | Datasource y dashboard. | Paneles con datos. |
| Validar alertas | Reglas cargadas y Alertmanager accesible. | Alertas agrupadas y listas para SMTP. |

Fuente: elaboración propia.

5.9.1. Seguimiento del primer despliegue en EC2

Durante el primer arranque en EC2 no hay que asumir que Docker ya ha terminado. El user data todavía puede estar instalando paquetes, creando swap, descargando imágenes o construyendo contenedores. Por eso docker ps puede fallar, mostrar poco o no reflejar el estado final hasta que el bootstrap acabe.

Para ver cómo va cargando el despliegue se entra por SSH o Session Manager y se ejecuta: `cat /var/log/taller-ec2-bootstrap.log`. Si se quiere verlo en directo, el comando más cómodo es `sudo tail -f /var/log/taller-ec2-bootstrap.log`. El log muestra pasos numerados del bootstrap y, si algo falla, incluye diagnósticos de docker compose ps -a y logs de servicios.

El bootstrap añade ec2-user al grupo docker. Ese cambio no se aplica a una terminal que ya estaba abierta. Después de terminar el bootstrap hay que salir con exit y abrir una nueva sesión SSH, o ejecutar newgrp docker. Hasta hacer eso, docker ps puede pedir sudo aunque el usuario ya esté añadido al grupo. Las contraseñas de producción no son las de la documentación local. En EC2 el bootstrap o el administrador rotan los secretos en /opt/taller_mecanico_asir/.env. Para consultar la contraseña de Grafana se usa `cd /opt/taller_mecanico_asir` y después `grep GRAFANA .env`. Para revisar Prometheus se usa `grep PROMETHEUS .env`: en este compose Prometheus no trae usuario y contraseña propios, por lo que se protege con Security Group o proxy; si en producción se añade autenticación por proxy, su variable también debe quedar en .env. Para otros servicios se usa el mismo patrón: `grep MYSQL .env` para MySQL, `grep SIMULATOR .env` para el token del simulador, `grep ALERTMANAGER .env` para la UI de alertas o `grep SMTP .env` para correo.

Figura 6. Flujo operativo del despliegue EC2 y consulta de credenciales.

```
```mermaid
flowchart TD
 crearInstancia["Crear instancia EC2"] --> userData["Ejecutar user data"]
 userData --> instalarDocker["Instalar Docker Compose y Buildx"]
 instalarDocker --> clonarRepo["Clonar repo en opt taller_mecanico_asir"]
 clonarRepo --> crearEnv["Crear .env y rotar secretos"]
 crearEnv --> deploy["Ejecutar deploy_aws_docker.sh"]
 deploy --> logsBootstrap["Consultar var log taller ec2 bootstrap log"]
 deploy --> nuevaSesion["exit y nueva SSH para grupo docker"]
 nuevaSesion --> dockerPs["docker ps y docker compose ps"]
```

```
crearEnv --> credenciales["grep GRAFANA .env y grep MYSQL .env"]
dockerPs --> navegador["Abrir puertos 80 3000 9090 9093 8890"]
...
```

Fuente: elaboración propia a partir de `scripts/ec2-user-data-bootstrap.sh` y `scripts/deploy_aws_docker.sh`.

## 5.10. Copias de seguridad y restauración

La base de datos es el activo crítico del sistema. Contiene usuarios, citas y contenidos publicados. En un entorno real, la pérdida de la base de datos implicaría pérdida de agenda y datos de clientes. Por ello, la memoria propone backups SQL y snapshots del volumen EBS.

En despliegue Docker local se pueden realizar backups con `mysqldump` dentro del contenedor MySQL. En producción se recomienda automatizar el proceso, comprimir el resultado y copiarlo a almacenamiento externo cifrado, como S3. La copia debe probarse mediante restauración, no basta con comprobar que el fichero existe.

La restauración debe hacerse con el servicio MySQL en ejecución y con credenciales correctas. Si se restaura sobre una base ya inicializada, pueden aparecer conflictos de claves duplicadas. Para una restauración limpia, conviene recrear el volumen o importar sobre una base vacía. Esta diferencia se documenta porque afecta directamente al procedimiento de recuperación ante desastre.

Los ficheros de imágenes subidos por administradores también deben respaldarse. En el `compose` de producción se almacenan en un volumen nombrado. Si solo se copia la base de datos y se pierden las imágenes, las noticias podrían quedar con enlaces rotos. Por tanto, el plan de backup debe cubrir base de datos y assets persistentes.

## 5.11. Despliegues alternativos: Coolify y Dokploy

Además de AWS, el repositorio contempla despliegues en plataformas de tipo PaaS autogestionado como Coolify y Dokploy. Estas herramientas simplifican la publicación desde repositorio Git, gestión de dominios, HTTPS y despliegues



Docker Compose. Son especialmente útiles para pequeños proyectos donde no se quiere operar toda la infraestructura de forma manual.

Coolify permite separar la aplicación y la monitorización en recursos distintos. Esta separación evita conflictos de puertos y facilita exponer Grafana, Prometheus o Alertmanager mediante rutas controladas. La documentación recomienda no publicar puertos fijos al host cuando la plataforma ya ofrece domains/routes.

Dokploy sigue una filosofía parecida: despliegue desde repositorio, compose específico y variables de entorno gestionadas desde panel. En ambos casos se mantiene la necesidad de rotar secretos, proteger métricas y limitar acceso a herramientas administrativas.

La existencia de varias rutas de despliegue aporta valor académico porque demuestra que la aplicación no depende de una única plataforma. El administrador puede elegir entre un entorno local, un VPS/EC2 clásico o una plataforma de despliegue con panel, según coste, experiencia y necesidades de la empresa.

**Tabla 11. Comparativa de escenarios de despliegue**

Escenario	Ventaja principal	Riesgo o limitación
Docker local	Rápido para desarrollo y pruebas completas.	No es una exposición pública profesional.
AWS EC2 + Docker	Control completo de instancia, red, disco y seguridad.	Mayor responsabilidad operativa.
Coolify	Gestión cómoda de dominios, rutas y despliegues.	Dependencia de la plataforma instalada.
Dokploy	Despliegue Git/Compose con panel y dominios.	Requiere configurar correctamente secretos y persistencia.
XAMPP local	Sencillo para pruebas PHP/MySQL en Windows.	No incluye observabilidad ni reproduce producción.

*Fuente: elaboración propia.*

## 5.12. Operación diaria y mantenimiento

Una vez implantado el sistema, el trabajo de ASIR continúa con operación y mantenimiento. El administrador debe revisar dashboards, comprobar alertas, actualizar imágenes, rotar credenciales y validar backups. La documentación del proyecto incluye comandos de logs, reinicio de servicios y troubleshooting.

La operación diaria puede estructurarse en tres niveles. El primer nivel revisa disponibilidad: aplicación responde, Prometheus scrapea y Grafana muestra datos. El segundo nivel revisa rendimiento: latencia, errores 5xx, conexiones de MySQL, CPU, memoria y disco. El tercer nivel revisa negocio: número de citas, crecimiento de usuarios y actividad de contenidos.

La política de actualización debe ser conservadora. En desarrollo se pueden probar versiones recientes, pero en producción conviene fijar tags concretos, validar cambios en un entorno de pruebas y aplicar actualizaciones con ventana de mantenimiento. Las actualizaciones de PHP, MySQL, Grafana y Prometheus deben revisarse por notas de seguridad.

La revisión de logs es complementaria a las métricas. Las métricas indican tendencias y estados; los logs explican detalles de una petición o error. En el proyecto, los logs HTTP y de tiempos alimentan el exporter PHP, mientras que los logs de contenedores permiten investigar fallos de arranque, conexión o permisos.

**Tabla 12. Plan de mantenimiento preventivo**

Periodicidad	Tarea	Herramienta
Diaria	Comprobar disponibilidad, errores HTTP y alertas activas.	Grafana, Prometheus, Alertmanager.
Semanal	Revisar espacio en disco, crecimiento de logs y estado de backups.	Node Exporter, Docker, scripts de backup.
Mensual	Aplicar actualizaciones menores probadas y revisar contraseñas expuestas.	Docker Compose, gestor de secretos.

Periodicidad	Tarea	Herramienta
Trimestral	Probar restauración de backup en entorno aislado.	MySQL, volúmenes Docker, S3 futuro.
Anual	Revisar coste cloud, dominio, certificados y arquitectura.	AWS Billing, documentación interna.

*Fuente: elaboración propia.*

## 6. RECURSOS Y PRESUPUESTO

### 6.1. Recursos utilizados

Tabla 13. Recursos utilizados

Recurso	Uso en el proyecto	Licencia o coste
PHP 8.2 y Apache	Ejecución de la aplicación web.	Software libre.
MySQL 8.0	Persistencia de datos.	Software libre.
Docker y Docker Compose	Contenerización y orquestación local/cloud.	Software libre con Docker Desktop según condiciones de uso.
Prometheus	Recopilación de métricas y reglas de alerta.	Software libre.
Grafana	Visualización de métricas.	Edición OSS o Cloud según despliegue; en el proyecto se usa contenedor local.
Alertmanager	Envío y agrupación de alertas.	Software libre.
AWS EC2 y EBS	Infraestructura cloud propuesta para producción.	Servicio de pago por uso.
Packer	Construcción de AMI base con Docker.	Software libre.
Git/GitHub	Control de versiones y almacenamiento del repositorio.	Cuenta gratuita o educativa.

Fuente: elaboración propia.

### 6.2. Presupuesto anual orientativo

El presupuesto se calcula para una pequeña empresa con una única instancia EC2 ejecutando aplicación, base de datos y monitorización. Los costes son orientativos y pueden variar por región, tráfico, tipo de instancia y política vigente

del proveedor. Para una primera implantación se considera suficiente una arquitectura sencilla; si el servicio crece, se recomienda separar base de datos y monitorización.

**Tabla 14. Presupuesto anual orientativo**

Concepto	Cantidad	Coste unitario estimado	Coste anual
EC2 t3.small 24/7	12 meses	25 EUR/mes	300 EUR
EBS gp3 cifrado 50 GiB	12 meses	5 EUR/mes	60 EUR
Dominio propio	1 año	12 EUR/año	12 EUR
Backups/snapshots	12 meses	5 EUR/mes	60 EUR
Correo SMTP para alertas	12 meses	0-10 EUR/mes	60 EUR
Implantación inicial	10 horas	25 EUR/hora	250 EUR
Mantenimiento preventivo	12 horas/año	25 EUR/hora	300 EUR
Contingencia 10%	1	Sobre subtotal	104 EUR
Total estimado	-	-	1.146 EUR

*Fuente: elaboración propia.*

El coste podría reducirse usando instancias gratuitas o créditos educativos, pero para una empresa se ha preferido un cálculo conservador. No se incluye IVA ni posibles costes de soporte externo. Si se añade un Application Load Balancer, CloudWatch intensivo o RDS MySQL, el coste anual aumentaría, aunque también mejoraría la operación.

## 7. VALIDACIÓN Y PRUEBAS

### 7.1. Pruebas funcionales

Las pruebas funcionales se organizan por perfiles. Como visitante se comprueba la navegación, visualización de noticias, registro y reserva como invitado. Como usuario registrado se verifica login, actualización de perfil y visualización de citas propias. Como administrador se prueban las operaciones CRUD y la protección de accesos.

La reserva de citas se valida con varios escenarios: fecha pasada, hora ocupada, datos incompletos, usuario autenticado y cliente invitado. La API debe responder con códigos adecuados y mensajes comprensibles. La interfaz actualiza los horarios ocupados mediante fetch para evitar que el usuario seleccione tramos no disponibles.

Las pruebas de subida de imágenes incluyen archivos válidos y no válidos. Se comprueba extensión, tipo MIME, tamaño máximo y escritura en el directorio persistente. En Docker, el volumen de imágenes debe conservarse tras reiniciar contenedores.

### 7.2. Pruebas de despliegue y monitorización

La validación de despliegue limpio consiste en eliminar volúmenes de desarrollo, reconstruir imágenes, levantar servicios y comprobar que MySQL queda healthy antes de que la aplicación web arranque. Esta verificación evita fallos típicos en los que Apache inicia antes de que la base de datos acepte conexiones.

Para Prometheus se revisan los targets y se confirma que php-app, mysqld-exporter, node-exporter y blackbox-exporter están en estado UP. En Grafana se comprueba que el datasource Prometheus se provisiona automáticamente y que el dashboard muestra métricas de aplicación, base de datos, sistema y negocio.

El simulador de tráfico se prueba con cargas breves, verificando que las líneas aparecen en logs/metrics.log y que Grafana diferencia source=app de source=simulator. Este punto es relevante porque permite demostrar observabilidad bajo tráfico sintético sin alterar el código de la aplicación.

## 7.3. Matriz de pruebas

La siguiente matriz resume pruebas recomendadas para la defensa y para una validación técnica previa. No sustituye a una suite automatizada completa, pero permite demostrar los flujos principales ante el tribunal y detectar fallos críticos antes de la exposición.

**Tabla 15. Matriz de pruebas funcionales y técnicas**

Área	Prueba	Resultado esperado
Registro	Crear usuario con correo válido, teléfono español y código postal de 5 dígitos.	Usuario creado y redirección a login.
Registro	Intentar registrar correo duplicado.	Mensaje de error sin insertar duplicados.
Login	Acceder con credenciales correctas.	Sesión iniciada y regeneración de ID.
Login	Acceder con contraseña incorrecta.	Mensaje de error y sin sesión.
Roles	Usuario normal intenta entrar en administración.	Redirección o bloqueo.
Citas	Consultar disponibilidad de un mes.	JSON con horarios ocupados.
Citas	Reservar hora libre como invitado.	Cita creada con datos de contacto.
Citas	Reservar hora ya ocupada.	Respuesta de conflicto.
Noticias	Subir imagen JPG/PNG válida.	Noticia publicada y fichero persistente.
Noticias	Subir fichero no permitido.	Error de validación.

Área	Prueba	Resultado esperado
Docker	Arranque limpio con volumen vacío.	MySQL inicializa esquema y web responde.
Métricas	Acceder a /metrics.php.	Métricas Prometheus en texto plano.
Prometheus	Revisar targets.	Targets principales en UP.
Grafana	Abrir dashboard principal.	Paneles con datos.
Alertas	Simular caída de endpoint.	Regla de alerta activa tras el tiempo definido.
Simulador	Ejecutar carga corta.	Aumentan métricas con source=simulator.

Fuente: elaboración propia.

## 7.4. Plan de demostración para la defensa

La exposición puede organizarse en tres bloques. En primer lugar, se presenta el problema del taller y la solución funcional: navegación, registro, login, cita y panel de administración. Este bloque debe ser breve y centrado en el valor para la empresa.

En segundo lugar, se muestra la implantación: docker compose ps, servicios levantados, base de datos, volúmenes y redes. Conviene explicar por qué MySQL no debe exponerse y cómo el servicio web depende de healthchecks. Esta parte conecta directamente con competencias ASIR.

En tercer lugar, se demuestra observabilidad: endpoint de métricas, Prometheus targets, dashboard Grafana y simulador de tráfico. Una carga breve permite ver cómo cambian las gráficas, diferenciando tráfico real y simulado. Esta demostración aporta evidencia visual de que la monitorización no es decorativa.



Como cierre, se explican las mejoras futuras: RDS, CI/CD, HTTPS definitivo, backups en S3 y políticas RGPD. Estas mejoras muestran criterio profesional y reconocen límites razonables del proyecto dentro del tiempo disponible.

**Tabla 16. Guion de demostración**

Minuto	Contenido	Evidencia
0-5	Contexto, objetivos y módulos ASIR implicados.	Portada, introducción y tabla de módulos.
5-12	Aplicación: login, citas, noticias y administración.	Navegador con flujos principales.
12-18	Infraestructura Docker y base de datos.	docker compose ps, esquema y volúmenes.
18-24	Monitorización y simulador.	Prometheus, Grafana y carga sintética.
24-28	AWS, seguridad y presupuesto.	Diagrama de despliegue y tabla de costes.
28-30	Conclusiones y mejoras.	Tabla de objetivos y evolución propuesta.

*Fuente: elaboración propia.*

## 8. CONCLUSIONES

### 8.1. Grado de consecución de objetivos

Tabla 17. Grado de consecución de objetivos

Objetivo	Grado	Observaciones
Aplicación web funcional para taller	Alto	Registro, login, citas, noticias, consejos y administración están contemplados.
Modelo de datos relacional	Alto	Tablas normalizadas, claves foráneas, índices y datos iniciales.
Despliegue reproducible	Alto	Docker Compose local, variantes AWS/Coolify/Dokploy y documentación asociada.
Monitorización integrada	Alto	Prometheus, Grafana, Alertmanager, exporters y blackbox.
Simulación de tráfico	Medio-Alto	Worker y UI separados; requiere entorno Docker y configuración segura de token.
Preparación para producción	Medio	Existe compose AWS, pero faltaría desplegar dominio real, TLS definitivo y backups automáticos en S3.

Fuente: elaboración propia.

### 8.2. Problemas encontrados

Tabla 18. Problemas encontrados y resolución

Problema	Impacto	Solución aplicada o propuesta
Orden de arranque entre web y MySQL	La aplicación podía iniciar antes de que la base de datos estuviera lista.	Healthcheck de MySQL y dependencia condition: service_healthy.

Problema	Impacto	Solución aplicada o propuesta
Rutas del exporter PHP al copiarse a la imagen	El endpoint de métricas podía fallar si las rutas relativas no coincidían.	Ajuste de rutas y validación de /metrics.php.
Compatibilidad de Node Exporter en Windows	Algunos montajes Linux no existen en Windows.	Documentación de limitaciones y configuración compatible en desarrollo.
Exposición de herramientas de monitorización	Riesgo si Grafana/Prometheus quedan públicos sin control.	En AWS se publican solo en loopback y se accede por túnel o proxy seguro.
Simulador de tráfico potencialmente abusivo	Podría usarse contra URLs externas sin autorización.	Token, confirmación de externos, bloqueo de privados y SSL verify por defecto.

Fuente: elaboración propia.

### 8.3. Mejoras futuras

Tabla 19. Propuestas de mejora

Mejora	Recursos necesarios	Beneficio esperado
Migrar MySQL a Amazon RDS	Instancia RDS, ajustes de variables y política de backups.	Menor carga operativa y backups gestionados.
Automatizar CI/CD con GitHub Actions	Runner, secretos, SSH/SSM o CodeDeploy.	Despliegues más repetibles y auditables.
HTTPS definitivo con ALB/ACM o Caddy	Dominio, certificado y configuración de proxy.	Confidencialidad y cumplimiento básico para datos personales.
Backups en S3 con retención	Bucket cifrado, lifecycle policy y script programado.	Recuperación ante pérdida de datos.
Notificaciones de cita por email robustas	Proveedor SMTP transaccional y plantillas.	Mejor experiencia del cliente.

Mejora	Recursos necesarios	Beneficio esperado
Calendario avanzado	Desarrollo adicional de interfaz y reglas de negocio.	Horarios especiales, festivos, duración variable y cancelaciones.
Endurecimiento RGPD	Política de privacidad, retención, exportación y borrado.	Mayor adecuación legal al tratar datos personales.

*Fuente: elaboración propia.*

## **9. ANEXOS Y DOCUMENTOS COMPLEMENTARIOS**

### **9.1. Índice de anexos**

- Anexo A. Inventario de archivos principales del repositorio.
- Anexo B. Comandos de despliegue local y AWS.
- Anexo C. Variables de entorno relevantes.
- Anexo D. Fragmentos de configuración y observabilidad.
- Anexo E. Referencias y bibliografía.

## Anexo A. Inventario de archivos principales

**Tabla 20. Inventario de artefactos del repositorio**

Ruta	Descripción
README.md	Descripción general, funcionalidades, credenciales de desarrollo y estado del proyecto.
database/database.sql	Script de creación de base de datos, tablas, índices y usuario admin inicial.
config/database.php	Conexión PDO y soporte de variables de entorno para local, Docker y proveedores.
includes/functions.php	Funciones comunes de sesión, roles, validación, CSRF y ficheros.
api/citas_api.php	API JSON para disponibilidad y creación de citas.
docker-compose.yml	Stack local completo con aplicación, base de datos, monitorización y simulador opcional.
docker-compose.aws.yml	Stack orientado a producción en EC2.
monitoring/prometheus/prometheus.yml	Configuración de scraping Prometheus.
monitoring/prometheus/alerts.yml	Reglas de alertas técnicas y de aplicación.
monitoring/grafana/dashboards/taller-mecanico-dashboard.json	Dashboard principal de Grafana.
docs/AWS_DOCKER_DEPLOYMENT.md	Guía de despliegue AWS con EC2, Docker Compose y Packer.
docs/TRAFFIC_SIMULATOR.md	Documentación del simulador de tráfico y su UI separada.

*Fuente: elaboración propia.*

## Inventario actualizado por carpetas

La raíz contiene las páginas PHP visibles por el usuario y por el administrador. La carpeta admin agrupa el panel interno; api contiene endpoints JSON; config centraliza entorno y conexión a base de datos; includes contiene cabecera, pie, funciones comunes y registro de métricas; database guarda esquema, migraciones y datos semilla.

La carpeta docker contiene entypoints, backup y las dos imágenes del simulador: el worker con Apache JMeter y la UI Apache/PHP. La carpeta monitoring contiene Prometheus, Grafana, Alertmanager, exporters, Telegraf y cAdvisor. La carpeta scripts agrupa despliegue AWS, verificaciones, herramientas de base de datos, scraping de noticias y ejecución de JMeter. La carpeta tests mantiene pruebas PHP de acceso y del simulador.

La documentación se reparte en README.md y docs. Las guías más importantes para operación son DOCKER\_DEPLOYMENT.md para local, AWS\_DOCKER\_DEPLOYMENT.md para EC2, MONITORING\_SETUP\_GUIDE.md para observabilidad y TRAFFIC\_SIMULATOR.md para JMeter. Los ficheros docker-compose.coolify\*.yml y docker-compose.dokploy.yml documentan despliegues alternativos en plataformas con panel.

## **Anexo B. Comandos de despliegue**

### **Comandos útiles en EC2 tras el bootstrap**

```
cd /opt/taller_mecanico_asir
cat /var/log/taller-ec2-bootstrap.log
grep GRAFANA .env
grep MYSQL .env
grep PROMETHEUS .env
grep SIMULATOR .env
exit
docker ps
docker compose --env-file .env -f docker-compose.aws.yml ps -a
```

Estos comandos no son pasos decorativos: sirven para comprobar que el despliegue ha terminado, encontrar contraseñas reales de producción y confirmar que la sesión SSH ya reconoce los permisos del grupo docker.

Despliegue local con Docker: copiar `.env.example` a `.env`, ajustar credenciales y ejecutar `docker compose up -d --build`. La aplicación queda disponible en el puerto configurado por `WEB_PORT` y Grafana en `GRAFANA_PORT`.

Despliegue con simulador: ejecutar `docker compose --profile traffic up -d` y acceder a la UI en el puerto configurado por `TRAFFIC_SIMULATOR_UI_PORT`. Antes de usarlo debe cambiarse `SIMULATOR_CONTROL_TOKEN`.

Despliegue AWS: crear instancia EC2, usar el user data scripts/ec2-user-data-bootstrap.sh o una AMI preparada, clonar el repositorio en `/opt/taller_mecanico_asir`, revisar `.env`, rotar secretos y ejecutar `./scripts/deploy_aws_docker.sh`. El bootstrap escribe su avance en `/var/log/taller-ec2-bootstrap.log` y el script imprime al final las URLs de navegador de Grafana, Prometheus, Alertmanager y la UI del simulador.

## Anexo C. Variables de entorno relevantes

**Tabla 21. Variables principales de configuración**

Variable	Uso
DB_HOST, DB_NAME, DB_USER, DB_PASS	Conexión de la aplicación PHP a la base de datos.
MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD	Inicialización y credenciales de MySQL en Docker.
APP_ENV, APP_DEBUG	Control de entorno y salida de errores.
GRAFANA_ADMIN_USER, GRAFANA_ADMIN_PASSWORD	Credenciales iniciales de Grafana.
ALERT_EMAIL_TO, SMTP_*	Configuración de correo para Alertmanager.
WEB_PORT, PROMETHEUS_PORT, GRAFANA_PORT, WEB_HOST_PORT, PROMETHEUS_HOST_PORT, GRAFANA_HOST_PORT,	Puertos publicados en local y en AWS. En AWS se abren desde el Security Group y se recomienda restringir



Variable	Uso
ALERTMANAGER_HOST_PORT, TRAFFIC_SIMULATOR_UI_HOST_PORT	MONITORING_SG_CIDR a la IP del administrador.
SIMULATOR_CONTROL_TOKEN	Token de control del simulador de tráfico. En EC2 se genera o se define en /opt/taller_mecanico_asir/.env y no se muestra al navegador.
SIM_EXTERNAL_TARGETS_ENABLED, SIM_SSL_VERIFY	Política de seguridad del simulador.

Fuente: elaboración propia.

## Anexo D. Fragmentos de configuración y observabilidad

Los siguientes fragmentos resumen decisiones relevantes. Se recomienda consultar los archivos originales del repositorio para ver el contenido completo y actualizado.

Docker Compose local define web, mysql, prometheus, grafana, alertmanager, node-exporter, mysqld-exporter, blackbox-exporter, cadvisor, telegraf y servicios opcionales de simulación. Los volúmenes conservan datos, logs e imágenes, y las redes separan aplicación y monitorización.

Prometheus obtiene métricas de /metrics.php, exporters y probes blackbox. Las reglas de alertas incluyen caída de métricas de aplicación, errores 5xx, latencia elevada, base de datos no saludable, conexiones MySQL altas, recursos de sistema y fallo de probes HTTP.

## 10. REFERENCIAS Y BIBLIOGRAFÍA

Amazon Web Services. Documentación oficial de Amazon EC2, Amazon EBS, Elastic Load Balancing y AWS Systems Manager. <https://docs.aws.amazon.com/>

Apache Software Foundation. Apache HTTP Server Documentation. <https://httpd.apache.org/docs/>

Docker Inc. Docker Documentation y Docker Compose Documentation. <https://docs.docker.com/>

Grafana Labs. Grafana Documentation. <https://grafana.com/docs/>

HashiCorp. Packer Documentation. <https://developer.hashicorp.com/packer/docs>

MySQL. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/>

PHP Group. PHP Manual: PDO, sessions, password hashing and security functions. <https://www.php.net/docs.php>

Prometheus Authors. Prometheus, Alertmanager, Node Exporter y Blackbox Exporter Documentation. <https://prometheus.io/docs/>

Repositorio del proyecto Taller Mecánico ASIR. Documentación interna: README.md, docs/DOCKER\_DEPLOYMENT.md, docs/AWS\_DOCKER\_DEPLOYMENT.md, docs/MONITORING\_SETUP\_GUIDE.md y docs/TRAFFIC\_SIMULATOR.md.